



Facultad de Ingeniería y Ciencias

Departamento de Ciencias de la Computación e Informática

Sistemas Inteligentes

Aprendizaje por Refuerzo para la Detección de Anomalías

Docente

Ricardo Gacitua Bustos

Integrante

Angelo Huaiquil Aravena

Módulo

ICC608

Temuco, Chile. Julio, 2026

1. Introducción al Aprendizaje por Refuerzo

El Aprendizaje por Refuerzo (Reinforcement Learning, RL) es la rama de la Inteligencia Artificial en la que un agente aprende a comportarse interactuando con un entorno, en vez de aprender a partir de ejemplos etiquetados de antemano. A diferencia del aprendizaje supervisado donde un modelo recibe pares entrada/salida correctos y minimiza un error de predicción en RL no existe una "respuesta correcta" explícita para cada situación: el agente prueba acciones, observa una señal de recompensa (positiva o negativa) y ajusta su comportamiento para maximizar la recompensa acumulada a largo plazo.

Esta capacidad resulta especialmente adecuada para problemas de decisión secuencial en entornos inciertos, donde la mejor acción depende del estado actual y sus consecuencias se manifiestan recién varios pasos después. La detección de anomalías en encontrar comportamientos que se desvían de lo esperado, sin conocerlos de antemano, encaja naturalmente en este marco cuando además existe una componente de navegación o búsqueda activa: no basta con clasificar una observación fija, hay que decidir dónde mirar a continuación.

2. Conceptos fundamentales

Todo problema de RL se describe con las mismas piezas:

- **Agente:** la entidad que aprende y decide. En este proyecto, un dron de monitoreo urbano.
- **Entorno:** el sistema con el que el agente interactúa y que responde a sus acciones. Aquí, una red vial modelada como grid de intersecciones.
- **Estado (s):** la información que el agente observa en un instante dado. En este caso, su posición dentro del grid.
- **Acción (a):** una decisión que el agente puede tomar en cada paso; aquí, una dirección de movimiento.
- **Recompensa (r):** la señal numérica, positiva o negativa, que el entorno devuelve tras cada acción, y que codifica qué tan deseable fue esa decisión.
- **Política (π):** la estrategia que mapea estados a acciones. Es lo que el agente efectivamente aprende: al inicio es casi aleatoria, y con la experiencia converge a un comportamiento cada vez más dirigido a maximizar la recompensa futura.

El agente y el entorno interactúan en un bucle continuo: el agente observa el estado, elige una acción según su política, el entorno transiciona a un nuevo estado y entrega una recompensa, y el ciclo se repite. Cada vuelta de este bucle es una oportunidad de aprendizaje.

3. Proceso de aprendizaje mediante recompensas

El agente no recibe instrucciones directas sobre qué hacer; solo recibe una señal escalar de recompensa después de cada acción. El aprendizaje consiste en usar esa señal para estimar, cada vez con más precisión, cuánto vale tomar una acción determinada en un estado determinado, el llamado valor de acción, $Q(s, a)$, y ajustar la política para favorecer las acciones con mayor valor esperado a largo plazo (no solo la recompensa inmediata).

Este proceso combina dos fuerzas en tensión: explotar el conocimiento ya adquirido (elegir la acción que hasta ahora parece mejor) y explorar alternativas todavía no probadas lo suficiente (porque podrían ser mejores de lo que el agente cree). Sin exploración, el agente podría quedar atrapado en la primera solución mediocre que encontró; sin explotación, nunca convergería a un comportamiento útil. El balance entre ambas se controla mediante el parámetro ϵ (épsilon), descrito en la sección siguiente.

4. Algoritmo utilizado: Q-learning tabular

Se utilizó Q-learning tabular, uno de los algoritmos clásicos de RL, por ser el más adecuado para un espacio de estados y acciones pequeño y discreto (25 estados $\times$ 4 acciones), y por permitir mostrar de forma completamente transparente, una tabla de números visible en pantalla, qué "sabe" el agente en cada momento, algo pedagógicamente valioso frente a un enfoque de caja negra como una red neuronal.

El algoritmo mantiene una tabla Q con una fila por estado y una columna por acción. Tras cada transición (s, a, r, s') se actualiza la entrada correspondiente con la ecuación de Bellman:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$$

donde:

- α (tasa de aprendizaje, 0.1): cuánto corrige cada nueva experiencia lo ya aprendido.
- γ (factor de descuento, 0.9): cuánto pesan las recompensas futuras frente a la inmediata.
- ϵ (tasa de exploración, 0.2): probabilidad de elegir una acción aleatoria en vez de la de mayor valor conocido (política ϵ -greedy).

Un detalle importante: la actualización usa el máximo Q del estado siguiente, no la acción que el agente realmente tomará después (que puede ser exploratoria). Esto hace de Q-learning un algoritmo off-policy: aprende sobre la política óptima incluso mientras se comporta de forma exploratoria.

5. Caso práctico: navegación de un dron para detectar anomalías de tráfico

Escenario: un dron de monitoreo patrulla una ciudad para encontrar accidentes o cortes de ruta (anomalías), evadiendo congestión, sin mapa ni ruta programada de antemano. Se eligió este escenario porque incorpora una geometría espacial real: una red vial es naturalmente un grid, lo que permite visualizar el aprendizaje como un agente que efectivamente se mueve sobre un mapa.

Modelado como MDP: grid de 5 $\times$ 5 intersecciones (25 nodos); el dron se mueve a una celda adyacente por paso (permanece en su lugar si el movimiento lo saca del grid). Cada celda se sorteja al inicio del episodio como anomalía (~18%), congestión (~15%) o normal, reveladas solo al visitarlas. Estado: posición del dron $\rightarrow$ 25 estados. Acciones: Arriba/Abajo/Izquierda/Derecha $\rightarrow$ 4 acciones. Recompensas: -1 por movimiento, -50 por entrar a congestión (cada vez), +100 por llegar a una anomalía por primera vez (revisitarla ya

no otorga el premio). Fin de episodio: todas las anomalías visitadas o 50 pasos, lo que ocurra primero. Este diseño traduce directamente las prioridades de la misión en una señal numérica, sin programar rutas ni reglas explícitas.

Visualización en vivo: la app muestra el mapa del grid coloreado por estado (verde=normal, amarillo=congestión, rojo=anomalía) con el dron desplazándose en tiempo real; un panel "por qué decidió esto" en cruz ($\uparrow\downarrow\leftarrow\rightarrow$) con los cuatro Q-values de la intersección actual y si la decisión fue una explotación clara, reñida, o una exploración (ϵ); la tabla Q completa (25×4 , colapsable); y una curva de recompensa con suavizado EMA y marcadores de episodio. En conjunto permiten observar cualitativamente el aprendizaje: con más episodios, el dron pasa de moverse erráticamente a rodear congestión y encontrar anomalías de forma directa. No se reportan métricas de convergencia formal, ya que no se ejecutó un benchmark estadístico controlado; la evidencia es observacional y en vivo.

6. Conclusiones y limitaciones

Lo que demuestra esta propuesta: un problema de navegación autónoma puede modelarse completamente como un proceso de decisión de Markov (posición $\rightarrow$ estado, dirección $\rightarrow$ acción, costo o beneficio del resultado $\rightarrow$ recompensa), y que el diseño de la recompensa por sí solo es suficiente para que, con una tabla Q de apenas 25×4 entradas, el agente desarrolle en pocas decenas de episodios una política visiblemente eficiente para encontrar anomalías evitando la congestión.

Limitaciones del enfoque:

- **Escala:** Q-learning tabular exige discretizar el estado. Una ciudad real con posición GPS continua y variables adicionales (velocidad, hora del día, sensores) haría explotar el tamaño de la tabla; escenarios de mayor dimensionalidad requerirían aproximación de funciones (p. ej. Deep Q-Networks) en vez de una tabla explícita.
- **Datos sintéticos:** el grid sortea anomalías y congestión de forma independiente por celda en cada episodio. El tráfico real presenta correlación espacial y temporal (un accidente afecta las calles vecinas, la congestión se desplaza con la hora pico) que este modelo simplificado no captura.
- **Entorno estático dentro de un episodio:** el grid no cambia mientras el dron lo recorre; en la realidad una anomalía puede aparecer o resolverse a mitad de un recorrido, lo que exigiría un entorno no estacionario y reentrenamiento continuo.
- **Optimalidad no garantizada:** al mantener ϵ y α fijos en vez de reducirlos con el tiempo, no existe garantía matemática de que la política aprendida sea la óptima exacta; en la práctica converge a un comportamiento muy eficiente, pero no a la ruta perfecta demostrable.

En conjunto, el proyecto cumple su objetivo pedagógico: mostrar de forma tangible cómo un agente de RL aprende una política de navegación a partir de recompensas, dejando explícitas las simplificaciones que separan esta demo de un sistema de producción para tráfico real.

7. Referencias bibliográficas

- Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.
- OpenAI. Spinning Up in Deep RL. <https://spinningup.openai.com/>
- Silver, D. (UCL / DeepMind). Course on Reinforcement Learning. University College London. <https://www.davidsilver.uk/teaching/>
- Juliani, A. The Beginner's Reinforcement Learning Playground. <https://awjuliani.github.io/web-rl-playground/> (referencia de diseño para el motor de visualización grid + agente de esta aplicación).